SQL Code

```sql
SELECT * FROM sales LIMIT 10;

-- Module 1: The Basics & Time Series
-- YoY sales and profit growth:

WITH yoy_growth AS(
SELECT
        EXTRACT(YEAR FROM order_date) AS current_year,
        SUM(sales) AS total_sales,
        SUM(profit) AS total_profit,

        LAG(SUM(sales)) OVER(ORDER BY EXTRACT(YEAR FROM order_date)) AS prev_year_sales,
        LAG(SUM(profit)) OVER(ORDER BY EXTRACT(YEAR FROM order_date)) AS prev_year_profit
FROM sales
GROUP BY EXTRACT(YEAR FROM order_date)
)
SELECT *,
        ROUND((total_sales - prev_year_sales) / prev_year_sales * 100,2) AS yoy_sales_pct,
        ROUND((total_profit - prev_year_profit) / prev_year_profit * 100,2) AS yoy_profit_pct
FROM yoy_growth;


-- Seasonality Weekend effect total sales and total return:

SELECT
        TO_CHAR(order_date,'Day') AS day_wise,
        SUM(sales) AS total_sales,
        SUM(CASE
                WHEN returned ='Yes' THEN 1 ELSE 0 END) AS total_returns
FROM sales
GROUP BY TO_CHAR(order_date,'Day')
ORDER BY total_sales DESC;


-- Best quarter year total sales:

WITH ranked_sales AS(
SELECT
        EXTRACT (YEAR FROM order_date) AS yearly,
        EXTRACT (QUARTER FROM order_date) AS quarterly,
        SUM(sales) AS total_sales
FROM sales
GROUP BY EXTRACT (QUARTER FROM order_date),EXTRACT (YEAR FROM order_date)
)
, sales_rn AS(
SELECT
```

```sql
        *,
        RANK() OVER(PARTITION BY yearly ORDER BY total_sales DESC) AS rn
FROM ranked_sales
)
SELECT
        yearly,
        quarterly,
        total_sales
FROM sales_rn
WHERE rn = 1
ORDER BY yearly;

-- Module 2: Profit Bleeding (Where are we losing money?)
-- Where we are losing money [Discount trap]:

SELECT
        category,
        sub_category,
        SUM(sales) AS total_sales,
        SUM(profit) AS total_profit,
        ROUND(AVG(discount)*100,2) As avg_discount_pct
FROM sales
GROUP BY category,
        sub_category
HAVING SUM(profit) < 0;


-- Return problems in regions with return pct:

SELECT
        region,
        SUM(sales) AS total_sales,
        SUM(profit) AS total_profit,
        COUNT(*) AS total_orders,
        SUM(CASE WHEN returned = 'Yes' THEN 1 ELSE 0 END) AS total_returns,
        ROUND(SUM(CASE WHEN returned = 'Yes' THEN 1 ELSE 0 END)::numeric * 100.0 /
COUNT(*),2) AS return_pct
FROM sales
GROUP BY region
ORDER BY return_pct DESC;


-- Top 10 loss making customers:

SELECT
        customer_id,
        customer_name,
        SUM(sales) AS total_sales,
        SUM(profit) AS total_loss
FROM sales
GROUP BY customer_id,customer_name
HAVING SUM(profit) <0
ORDER BY total_loss
LIMIT 10;
```

```sql

--Module 3: Advanced Supply Chain & Logistics
-- Shipping Delay Analysis:

SELECT
        ship_mode,
        --AVG(AGE(ship_date, order_date)) AS delay_days
        ROUND((AVG(ship_date - order_date)),2) AS delay_days
FROM sales
GROUP BY ship_mode
ORDER BY delay_days;


-- Late delivery Impact Compare on time vs late returns:

WITH shipping_data AS(
        SELECT
                order_id,
                returned,
                ship_date - order_date AS deliver_days
        FROM sales
)
SELECT
        CASE
                WHEN deliver_days > 3 THEN 'Late' ELSE 'On_time' END AS
delivery_status,
        SUM( CASE
                        WHEN returned = 'Yes' THEN 1 ELSE 0 END ) AS return_orders,
        COUNT(DISTINCT order_id) AS total_orders,
        ROUND(COUNT(DISTINCT CASE
                        WHEN returned = 'Yes' THEN 1 ELSE 0 END )*100.0/
COUNT(DISTINCT order_id)*100 ,2) AS return_pct
FROM shipping_data
GROUP BY CASE
                WHEN deliver_days > 3 THEN 'Late' ELSE 'On_time' END


-- Salesperson performance:

SELECT
        retail_sales_people,
        SUM(sales) AS total_sales,
        SUM(profit) AS total_profit,
        ROUND(SUM(profit) / SUM(sales) *100,2) AS profit_margin
FROM sales
GROUP BY retail_sales_people


-- (Window Functions & CTEs)
-- Churn customers : placed order in 2015 & 2016 but never placed in 2017:

WITH customer_details AS(
SELECT
        customer_id,
```

```sql
        EXTRACT(YEAR FROM order_date) AS years
FROM sales
)
SELECT DISTINCT customer_id
FROM customer_details
GROUP BY customer_id
HAVING COUNT(DISTINCT CASE WHEN years IN (2015, 2016) THEN years END) = 2
    AND
    COUNT(DISTINCT CASE WHEN years = 2017 THEN years END) = 0;


-- 30 days Moving average:

WITH daily_sales AS(
SELECT
        order_date,
        SUM(sales) AS daily_sales
FROM sales
GROUP BY order_date
)
SELECT
        order_date,
        daily_sales,
        AVG(daily_sales) OVER(ORDER BY order_date ROWS BETWEEN 29 PRECEDING AND
CURRENT ROW) AS moving_avg
FROM daily_sales


-- Customer Segmentation:

SELECT
        customer_id,
        customer_name,
        COUNT( customer_id) AS total_orders,
        SUM(sales) AS total_sales,
        CASE
                WHEN COUNT( customer_id)= 1 THEN 'One_time_buyer'
                WHEN SUM(sales) > 5000 THEN 'VIP'
                ELSE 'Regular'
        END AS segments
FROM sales
GROUP BY customer_id,
        customer_name
ORDER BY total_sales DESC;


-- MoM growth with pct

WITH month_growth AS(
SELECT
                DATE_TRUNC('month', order_date)::DATE AS month_date,
            EXTRACT(MONTH FROM order_date) AS month_number,
            TRIM(TO_CHAR(order_date, 'Mon')) AS month_name,
                SUM(sales) AS total_sales
```

```
FROM sales
GROUP BY DATE_TRUNC('month', order_date)::DATE,
                EXTRACT(MONTH FROM order_date),
                TRIM(TO_CHAR(order_date, 'Mon'))
)
SELECT
        *,
        LAG(total_sales) OVER(ORDER BY month_date) AS previous_month_sales,
        ROUND((total_sales - LAG(total_sales) OVER(ORDER BY month_date)) *100 /
LAG(total_sales) OVER(ORDER BY month_date),2)as pct
FROM month_growth


-- Most profitable route:

SELECT
        state,
        city,
        COUNT(order_id) as total_orders,
        SUM(profit) as total_profit,
        ROUND(SUM(profit) / COUNT(order_id),2) AS per_order_profit
FROM sales
GROUP BY state,
                city
HAVING COUNT(order_id) >=10
ORDER BY total_profit DESC;
```